PYTHON PROGRAMMING LAB SOLUTIONS

Comprehensive Algorithm Implementations & Flowchart Logic

1. ATM Transaction Sequence Simulation

Python Code:

```
def atm_sequence(balance, transactions):
    print(f"Initial Balance: ${balance}")
    for tx in transactions:
        tx_type, amount = tx
        if tx_type == "W":
            if amount <= balance:
                balance -= amount
                print(f"Withdraw ${amount} | Success | B
            else:
                print(f"Withdraw ${amount} | Failed (Ins
        elif tx_type == "D":
            balance += amount
            print(f"Deposit ${amount}  | Success | Bal:
    return balance

# Example Execution
initial = 1000
tx_list = [("W", 200), ("D", 500), ("W", 1500)]
final_bal = atm_sequence(initial, tx_list)
```

Flowchart Diagram:

```
( START ) | [ Input Balance, Transactions ] | / For
```

2. Advanced Numeric Pattern Generator

Python Code:

```
def generate_pattern(n):
    for i in range(1, n + 1):
        # Ascending sequence
        row = [str(j) for j in range(1, i + 1)]
        # Descending sequence
        row += [str(j) for j in range(i - 1, 0, -1)]
        # Center align with spacing
        print(" ".join(row).center(n * 4))

# Example Execution
generate_pattern(5)
```

Flowchart Diagram:

```
( START ) | [ Input Row Count n ] | [ i = 1 ] | < Is
```

DATA VERIFICATION & SECURITY MODULES

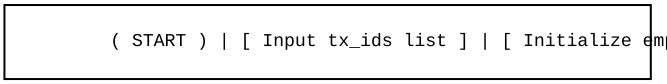
3. Repeated Transaction Detection

Python Code:

```
def detect_repeated_transactions(tx_ids):
    seen = set()
    duplicates = set()
    for tx in tx_ids:
        if tx in seen:
            duplicates.add(tx)
        else:
            seen.add(tx)
    return list(duplicates)

# Example Execution
transactions = ["TX101", "TX102", "TX101", "TX103", "TX
repeats = detect_repeated_transactions(transactions)
print("Repeated Transactions:", repeats)
```

Flowchart Diagram:



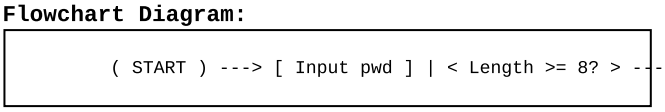
4. Password Policy Validation Engine

Python Code:

```
def validate_password(pwd):
    if len(pwd) < 8:
        return False, "Min 8 characters required"
    if not any(c.isupper() for c in pwd):
        return False, "At least one uppercase required"
    if not any(c.islower() for c in pwd):
        return False, "At least one lowercase required"
    if not any(c.isdigit() for c in pwd):
        return False, "At least one digit required"
    spec = "!@#$%^&*()-_+="
    if not any(c in spec for c in pwd):
        return False, "At least one special character re
    return True, "Password Valid"

# Example
status, msg = validate_password("SecurePass#123")
print(f"Status: {status} | Message: {msg}")
```

Flowchart Diagram:



TEXT PROCESSING & AUTOMATED EVALUATION

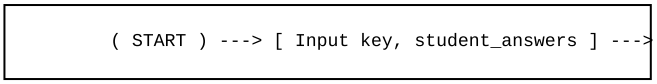
5. String-Based Examination Answer Evaluation

Python Code:

```
def evaluate_exam(key, student_answers):
    score = 0
    total = len(key)
    for k, a in zip(key, student_answers):
        if a.upper() == k.upper():
            score += 4
        elif a.upper() == "N":
            score += 0
        else:
            score -= 1
    percentage = (score / (total * 4)) * 100
    return score, percentage

# Example Execution
answer_key = ["A", "B", "C", "D", "A"]
student    = ["A", "B", "N", "A", "A"]
sc, pct = evaluate_exam(answer_key, student)
print(f"Score: {sc} | Percentage: {pct:.2f}%")
```

Flowchart Diagram:



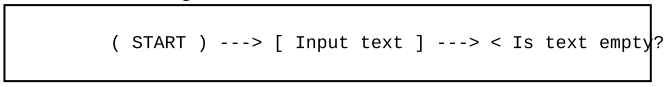
6. Text Compression Using Character Runs (RLE)

Python Code:

```
def compress_text(text):
    if not text:
        return ""
    compressed = []
    count = 1
    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            compressed.append(text[i - 1] + str(count))
            count = 1
    compressed.append(text[-1] + str(count))
    return "".join(compressed)

# Example Execution
input_str = "AAABBCDDDD"
res = compress_text(input_str)
print("Compressed Output:", res)  # Output: A3B2C1D4
```

Flowchart Diagram:



INTERACTIVE SIMULATION & NESTED ANALYSIS

7. ATM Menu-Driven Banking Simulation

Python Code:

```
def atm_menu():
    bal = 1000.0
    while True:
        print("\n1. Check Balance  2. Deposit")
        print("3. Withdraw       4. Exit")
        ch = input("Enter choice (1-4): ")
        if ch == '1':
            print(f"Balance: ${bal:.2f}")
        elif ch == '2':
            amt = float(input("Deposit Amt: "))
            bal += amt
        elif ch == '3':
            amt = float(input("Withdraw Amt: "))
            if amt <= bal:
                bal -= amt
            else:
                print("Insufficient Funds")
        elif ch == '4':
            print("Session Ended.")
            break

# atm_menu()
```

Flowchart Diagram:



8. Nested List Classroom Occupancy Analysis

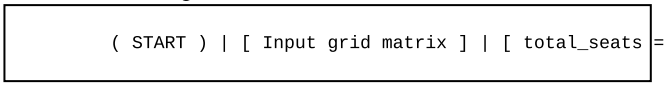
Python Code:

```
def analyze_occupancy(grid):
    total_seats = 0
    occupied_seats = 0
    for r_idx, row in enumerate(grid):
        r_occ = row.count(1)
        r_tot = len(row)
        total_seats += r_tot
        occupied_seats += r_occ
        rate = (r_occ / r_tot) * 100
        print(f"Row {r_idx+1}: {r_occ}/{r_tot} ({rate:.1

    overall_rate = (occupied_seats / total_seats) * 100
    print(f"Total Occupancy: {occupied_seats}/{total_sea

# Example (1 = Occupied, 0 = Empty)
classroom = [[1, 0, 1, 1], [0, 1, 0, 0], [1, 1, 1, 1]]
analyze_occupancy(classroom)
```

Flowchart Diagram:



MATRIX PROCESSING & CONFLICT DETECTION

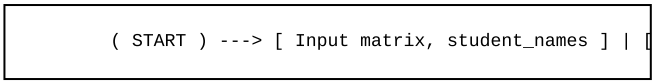
9. Student Marks Matrix Analysis

Python Code:

```
def analyze_marks(matrix, student_names):
    for i, row in enumerate(matrix):
        total = sum(row)
        avg = total / len(row)
        p_f = "PASS" if all(m >= 40 for m in row) else "
        print(f"{student_names[i]}: Total={total}, Avg=

# Matrix: [Math, Physics, Chemistry]
marks = [
    [85, 78, 90],
    [35, 60, 50],
    [92, 88, 95]
]
names = ["Alice", "Bob", "Charlie"]
analyze_marks(marks, names)
```

Flowchart Diagram:



10. Tuple-Based Employee Shift Conflict Detection

Python Code:

```
def detect_shift_conflicts(shifts):
    conflicts = []
    n = len(shifts)
    for i in range(n):
        for j in range(i + 1, n):
            emp1, s1, e1 = shifts[i]
            emp2, s2, e2 = shifts[j]
            # Check overlap: start1 < end2 and start2 <
            if s1 < e2 and s2 < e1:
                conflicts.append((emp1, emp2, (max(s1, s
    return conflicts

# Shift format: (Employee_ID, Start_Hour, End_Hour)
shift_schedule = [("E101", 8, 12), ("E102", 10, 14), ("E
conf = detect_shift_conflicts(shift_schedule)
print("Detected Conflicts:", conf)
```

Flowchart Diagram:

